

SymptoScan

“Smart Symptom Analysis for Better Health Decisions.”
(Symptoms Based Disease Prediction & Consultation System)

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Submitted By:

Jainam Shah 202512012
Jay Shah 202512059
Priya J Shah 202512100
Priya D Shah 202512105

Submitted To:

Prof. Jayprakash Lalchandani
Course Instructor - Software Engineering (IT632), DAU

1. INTRODUCTION

1.1. Purpose

This document outlines the specifications for a Disease Prediction and Consultation System, enabling users to predict diseases based on symptoms using machine learning, consult with doctors, and receive medical guidance.

1.2. Scope

- Predict diseases based on user-inputted symptoms
- Connect patients with doctors for consultation
- Track patient history and consultation records
- Make secure appointments and payments
- Admin panel for system management
- Doctor and patient dashboards

1.3. Intended Audience

Stakeholders, developers, testers, designers, data scientists, healthcare professionals, and operations engineers involved in system development, deployment, and operation.

1.4. Product Features

- User registration & login (Patient, Doctor, Admin roles)
- Symptom-based disease prediction using ML
- Real-time doctor-patient chat
- Patient history tracking

- Feedback and rating system
- Admin dashboard with reports
- Doctor profile management
- Notification system
- Secure authentication with JWT
- Report generation and analytics

1.5. Non-Functional Requirements

- Performance: App loads in under 2 seconds, ML prediction in < 1 second
- Security & Privacy: JWT authentication, HTTPS, HIPAA compliant data handling
- Maintainability: Modular codebase with clear separation of concerns
- Usability: Intuitive and responsive UI/UX
- Reliability: 99.9% uptime
- Scalability: Elastic infrastructure to handle growing users
- Accuracy: ML model accuracy > 85%

1.6. Architecture Overview

- Frontend: React.js
- Backend: Django & Django REST Framework
- Database: PostgreSQL
- Machine Learning: Scikit-Learn, Pandas, NumPy
- Authentication: JWT-based (Djoser + DRF)
- Hosting: Cloud infrastructure (AWS/Azure)

1.7. Future Enhancements

- AI-powered health recommendations
- Telemedicine video consultation
- Integration with wearable devices
- Prescription management system
- Health insurance integration
- Multi-language support
- Mobile application (Flutter/React Native)

1.8. User Stories for Disease Prediction and Consultation System

ID	User Role	Wants To	So That
US001	Patient	register using email and password	I can securely access the platform
US002	Returning Patient	log in with my credentials	I can quickly access my previous records
US003	Doctor	register and complete my profile with specialization	I can receive consultation requests
US004	Patient	select symptoms from a comprehensive list	I can get an accurate disease prediction
US005	Patient	receive disease predictions based on my symptoms	I know what condition I might have
US006	Patient	view my prediction history	I can track my health over time
US007	Patient	see a list of available doctors	I can choose whom to consult
US008	Patient	initiate a chat with a selected doctor	I can get professional medical advice
US009	Doctor	view incoming consultation requests	I can manage my patient queue
US010	Doctor	accept or reject consultation requests	I can control my workload
US011	Doctor	view patient symptoms and prediction history	I can provide informed consultation
US012	Doctor	chat with patients in real-time	I can provide timely medical guidance
US013	Patient	provide feedback about doctors	I can help improve service quality
US014	Patient	rate doctors based on consultation experience	other patients can make informed choices
US015	Doctor	provide feedback about the platform	I can help improve the system
US016	Admin	view system statistics and reports	I can monitor platform health
US017	Admin	see total patients, doctors, and predictions	I can understand system usage
US018	Admin	view patient and doctor feedback	I can identify areas for improvement
US019	Admin	manage user accounts	I can maintain platform integrity
US020	Admin	monitor consultation activity	I can ensure quality service delivery
US021	Patient	receive notifications about consultation status	I stay informed about my requests
US022	Doctor	receive notifications when patients request consultation	I can respond promptly
US023	System	save all prediction and consultation history	users can access their medical records
US024	System	ensure secure data transmission	patient privacy is protected
US025	System	provide accurate ML predictions	users receive reliable health insights

1.9. Disease Prediction and Consultation System - Project Plan

Overview

- Project Duration: Approximately 8–10 weeks
- Development Methodology: Agile methodology using Scrum with 1-week sprints
- Each sprint corresponds to one week of development and review.

Phase 1: Requirement Analysis & Planning (Week 1)

Deliverables:

- Finalized Software Requirements Specification (SRS)
- Prioritized Feature List
- Technology Stack Decision
- Project Roadmap and Gantt Chart

Tasks:

- Stakeholder meetings, finalize user stories for work distribution
- Define MVP scope
- Setup GitHub, Figma, Jira, Slack
- Prepare data collection strategy for ML model

Phase 2: Design (Week 2)

Deliverables:

- Wireframes and UI Prototypes
- Database ER Diagram
- API Specification Document
- ML Model Architecture Design

Tasks:

- User flow design for patient, doctor, and admin
- Mockup creation and stakeholder review
- Database schema design
- ML pipeline architecture

Phase 3: Backend Development (Week 3–4)

Deliverables:

- REST APIs (Django REST Framework)
- JWT-based Authentication System
- PostgreSQL Database Integration
- Initial ML Model Training

Tasks:

- Setup repository and CI/CD pipeline
- Implement user management, authentication
- Implement disease prediction API
- Implement chat and consultation APIs
- Write unit tests
- Train ML model on symptom-disease dataset

Phase 4: Frontend Development (Week 5–6)

Deliverables:

- Web Application (React.js)
- Patient Dashboard
- Doctor Dashboard
- Admin Dashboard
- Symptom Input Interface
- Chat Interface

Tasks:

- API integration with backend
- Implement user flows (registration, login, prediction, consultation)
- Implement feedback and rating systems
- Responsive UI design
- Frontend testing

Phase 5: Integration & Testing (Week 7)

Deliverables:

- Integrated System
- ML Model Evaluation Report

- Test Cases Document
- Bug Fix Report

Tasks:

- End-to-end integration testing
- ML model accuracy testing and tuning
- Functional testing of all modules
- Security testing (JWT, HTTPS)
- Bug identification and fixing
- User acceptance testing preparation

Phase 6: DevOps & Deployment (Week 8)

Deliverables:

- Cloud deployment on AWS/Azure/GCP
- Dockerized services
- Monitoring dashboards
- CI/CD pipeline

Tasks:

- Setup production infrastructure
- Configure PostgreSQL database
- Deploy Django backend and React frontend
- Setup SSL certificates
- Implement automated backups
- Setup monitoring and logging

Phase 7: Launch MVP (Week 9)

Deliverables:

- Live web application
- Admin portal for system management
- Documentation (User Guide, API Docs)
- Training materials for doctors

Tasks:

- Final testing in production environment
- Onboard initial doctors
- Launch marketing campaign
- Setup customer support system
- Monitor system performance

Tools & Technologies

Area	Tools / Stack
Frontend	React.js
Backend	Django, Django REST Framework
Database	PostgreSQL
Machine Learning	Scikit-Learn, Pandas, NumPy
Authentication	JWT (Djoser)
DevOps	Docker, AWS/GCP, Jenkins/GitHub CI
Design	Figma
Project Management	Jira, Confluence
Communication	Slack, Google Meet
Testing	Pytest, Postman, Selenium, Jest

